

LỜI CẢM ƠN

Em xin chân thành cảm ơn thầy cô đã tận tình hướng dẫn và trang bị những kiến thức nền tảng quý báu để em hoàn thành bài tập lớn này. Con cũng gửi lời tri ân sâu sắc đến gia đình, những người luôn là điểm tựa vững chắc cả về vật chất lẫn tinh thần. Cảm ơn bạn bè đã đồng hành, hỗ trợ và cùng nhau vượt qua những giai đoạn áp lực nhất của dự án.

Đặc biệt, cảm ơn người yêu vì sự thấu hiểu, kiên nhẫn và luôn khích lệ để mình có thêm động lực phấn đấu. Cuối cùng, tôi muốn cảm ơn chính bản thân mình vì đã không bỏ cuộc, luôn giữ vững kỷ luật và sự tập trung cao độ trong suốt quá trình nghiên cứu. Sự nỗ lực và quyết tâm đó đã giúp tôi chuyển hóa những ý tưởng trên bản thảo thành kết quả trọn vẹn ngày hôm nay.

Đây không chỉ là một bài tập lớn, mà còn là cột mốc ý nghĩa cho sự trưởng thành của chính tôi.

Chương 1

GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong kỷ nguyên của tự động hóa và trí tuệ nhân tạo, khả năng di chuyển độc lập của các thực thể thông minh (như robot kho vận, thiết bị bay không người lái hay các nhân vật trong môi trường ảo) đã trở thành một yêu cầu thiết yếu. Thực tế cho thấy, việc di chuyển từ một điểm này đến một điểm khác không đơn thuần là một đường thẳng nối liền hai vị trí; đó là một quá trình liên tục đối mặt với các hệ thống vật cản phức tạp, đa dạng và các ràng buộc về không gian.

Tại các kho bãi thông minh hay các nhà máy sản xuất tự động, một sai sót nhỏ trong việc tính toán lộ trình có thể dẫn đến va chạm, gây thiệt hại lớn về tài sản hoặc làm đình trệ toàn bộ dây chuyền cung ứng. Thách thức đặt ra là làm thế nào để xác định được một lộ trình không chỉ khả thi (tránh được mọi vật cản) mà còn phải là lộ trình tối ưu nhất về mặt khoảng cách và thời gian xử lý trong một không gian lưới hai chiều đầy rẫy chướng ngại vật.

Việc giải quyết hiệu quả bài toán tìm đường này mang lại lợi ích trực tiếp cho các doanh nghiệp vận tải và logistics thông qua việc tối ưu hóa hiệu suất hoạt động của robot. Xa hơn thế, bài toán tìm đường trên lưới 2D còn đóng vai trò là xương sống cho nhiều lĩnh vực khác như: phát triển hệ thống bản đồ số (GPS), thiết kế trí tuệ nhân tạo trong trò chơi điện tử, và hỗ trợ công tác cứu hộ trong các môi trường nguy hiểm mà con người không thể tiếp cận. Một lời giải chính xác cho bài toán này chính là nền tảng để xây dựng những hệ thống tự hành thông minh, linh hoạt và an toàn hơn trong tương lai.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, bài toán tìm đường chủ yếu dựa vào hai hướng: duyệt mù như Dijkstra và tìm kiếm định hướng như A^* . Dijkstra đảm bảo độ chính xác nhưng lãng phí tài nguyên do duyệt toàn bộ không gian, trong khi các phương pháp tham lam tuy nhanh nhưng dễ đi vào ngõ cụt và thiếu tính tối ưu.

Hạn chế lớn nhất của các ứng dụng hiện tại là sự mất cân bằng giữa tốc độ xử lý và độ dài quãng đường, cùng việc thiếu giao diện trực quan để quan sát cơ chế ra quyết định.

Trên cơ sở phân tích đó, đề tài này tập trung vào việc nghiên cứu và hiện thực hóa thuật toán A^* để giải quyết bài toán tìm đường trong không gian lưới hai chiều có vật cản. Mục tiêu cốt lõi là tận dụng ưu điểm của hàm Heuristic để giới hạn phạm vi tìm kiếm, từ đó khắc phục triệt để nhược điểm duyệt rộng của các phương pháp cũ. Phần mềm được phát triển sẽ tập trung vào các chức năng chính bao gồm việc thiết lập bản đồ tùy biến, mô phỏng sinh động các bước lan tỏa của thuật toán từ tập mở đến tập đóng, và cuối cùng là xác định chính xác lộ trình ngắn nhất với chi phí thấp nhất. Thông qua việc kết hợp giữa lý thuyết toán học chặt chẽ và giao diện tương tác trực quan, đề tài hướng tới việc tạo ra một công cụ hỗ trợ hữu hiệu trong việc nghiên cứu và ứng dụng điều hướng thông minh.

1.3 Định hướng giải pháp

Đề tài giải quyết bài toán thông qua thuật toán A^* kết hợp với thư viện đồ họa Pygame của ngôn ngữ lập trình Python. Lựa chọn Python giúp tối ưu hóa việc triển khai cấu trúc dữ liệu và xử lý thuật toán một cách linh hoạt, đồng thời hỗ trợ xây dựng giao diện mô phỏng trực quan nhanh chóng.

Giải pháp cụ thể là phát triển một ứng dụng máy tính cho phép người dùng vẽ vật cản trực tiếp trên lưới bằng chuột. Hệ thống sẽ sử dụng cấu trúc hàng đợi ưu tiên (Priority Queue) để quản lý danh sách các nút, thực hiện tính toán hàm chi phí và cập nhật trạng thái màu sắc của các ô lưới (Open, Closed, Path) ngay trong quá trình thực thi để mô phỏng luồng chạy của thuật toán.

Đóng góp chính của bài tập lớn là cung cấp một công cụ trực quan hóa sinh động cơ chế hoạt động của thuật toán A^* , giúp chuyển đổi những lý thuyết toán học phức tạp thành hình ảnh dễ hiểu. Kết quả đạt được là một chương trình hoàn thiện có khả năng tìm đường tối ưu trong môi trường lưới có vật cản, đảm bảo tính ổn định và tốc độ phản hồi nhanh, hỗ trợ tốt cho việc học tập và nghiên cứu về trí tuệ nhân tạo.

1.4 Bố cục bài tập lớn

Nội dung báo cáo bài tập lớn được tổ chức thành 4 chương chính cùng các phần bổ trợ, cụ thể như sau:

Chương 2 cơ sở lý thuyết: Trình bày các kiến thức nền tảng về bài toán tìm đường đi ngắn nhất và nguyên lý hoạt động cốt lõi của thuật toán A^* . Phần này tập trung làm rõ cách sử dụng cấu trúc dữ liệu, cách tính toán hàm chi phí heuristic và chiến lược duyệt đỉnh để tránh vật cản trên không gian lưới 2 chiều.

Chương 3 thực nghiệm và đánh giá kết quả: Mô tả quá trình lập trình cài đặt thuật toán và xây dựng môi trường mô phỏng không gian lưới. Chương này sẽ đưa ra các kịch bản thử nghiệm với mật độ và vị trí vật cản khác nhau, từ đó phân tích và đánh giá hiệu suất của thuật toán A^* (thời gian chạy, bộ nhớ, độ tối ưu của đường đi).

Chương 4 kết luận và hướng phát triển: Tổng kết tính khả thi và độ hiệu quả của thuật toán đã triển khai so với mục tiêu ban đầu. Đồng thời, nhận định những hạn chế còn tồn tại và đề xuất hướng phát triển mở rộng, chẳng hạn như tối ưu hàm heuristic hoặc áp dụng cho môi trường có vật cản di động.

Chương 2

CƠ SỞ LÝ THUYẾT

Sau khi đã xác định được mục tiêu và bài toán cụ thể tại Chương 1, việc xây dựng một nền tảng lý thuyết vững chắc là bước đi tiên quyết để hiện thực hóa dự án. Nếu Chương 1 đóng vai trò dẫn dắt và mô tả bài toán tìm đường trên lưới 2D, thì Chương 2 sẽ tập trung vào việc giải quyết bài toán đó bằng các mô hình toán học và thuật toán chuyên sâu.

Sự cần thiết của chương này nằm ở việc làm rõ cơ chế vận hành của thuật toán A^* – trái tim của hệ thống. Việc hiểu đúng bản chất của các hàm chi phí và các biến thể Heuristic không chỉ giúp nhóm tối ưu hóa tốc độ tìm kiếm mà còn là cơ sở để đánh giá tính đúng đắn của chương trình sau khi triển khai thực tế.

Trong chương này, chúng ta sẽ lần lượt tìm hiểu các vấn đề trọng tâm thông qua các mục lớn sau:

- **Bài toán tìm đường:** Đồ thị và bài toán tìm đường trong lưới 2D có vật cản.
- **Thuật toán A^* :** Định nghĩa và đặc điểm của phương pháp tìm kiếm có thông tin (informed search), các hàm Heuristic, so sánh, quy trình hoạt động thuật toán.

2.1 Bài toán tìm đường

Để bắt đầu giải quyết bài toán tìm đường, trước hết cần tìm hiểu về cấu trúc dữ liệu nền tảng thường được sử dụng để mô hình hóa mạng lưới di chuyển: cấu trúc dữ liệu Đồ thị.

2.1.1 Khái niệm về Đồ thị (Graph)

Trong toán học và khoa học máy tính, đồ thị là một mô hình trừu tượng dùng để biểu diễn các mối quan hệ tương tác giữa các đối tượng trong một tập hợp. Một đồ thị được xác định bởi cặp tập hợp $G = (V, E)$, trong đó:

- V (Vertices/Nodes): Tập hữu hạn các đỉnh, đại diện cho các đối tượng hoặc các vị trí.
- E (Edges): Tập các cạnh, biểu diễn liên kết giữa các cặp đỉnh trong V . Mỗi cạnh $e \in E$ có thể mang tính chất hướng (cung) hoặc vô hướng.

Dựa trên đặc điểm của các cạnh và trọng số, đồ thị được phân loại thành các dạng phổ biến sau:

- **Đồ thị vô hướng (Undirected Graph):** Cạnh nối giữa hai đỉnh không phân biệt thứ tự, biểu diễn mối quan hệ hai chiều đối xứng.
- **Đồ thị có hướng (Directed Graph):** Các cạnh (cung) có hướng xác định, chỉ rõ lộ trình từ đỉnh đầu đến đỉnh cuối.
- **Đơn đồ thị (Simple Graph):** Giữa hai đỉnh bất kỳ chỉ tồn tại tối đa một cạnh và không có các vòng lặp tại một đỉnh (khuyên).
- **Đa đồ thị (Multigraph):** Cho phép tồn tại nhiều cạnh khác nhau cùng nối giữa hai đỉnh.
- **Đồ thị trọng số (Weighted Graph):** Mỗi cạnh được gán một giá trị số thực (trọng số), đại diện cho chi phí, khoảng cách hoặc thời gian di chuyển giữa hai đầu nút.

Trong bài toán tìm đường, chúng ta thường làm việc với các loại đồ thị phổ biến như: đồ thị vô hướng, đồ thị có hướng và đặc biệt là **đồ thị có trọng số** (Weighted Graph) – nơi mỗi cạnh được gán một giá trị biểu thị chi phí hoặc khoảng cách di chuyển.

2.1.2 Bài toán tìm đường (Pathfinding)

Bài toán tìm đường là quá trình xác định một lộ trình tối ưu (thường là ngắn nhất hoặc chi phí thấp nhất) từ một điểm xuất phát đến một điểm đích. Thuật toán phải vượt qua các ràng buộc về không gian, trọng số cạnh hoặc các vật cản để tìm ra kết quả hiệu quả nhất.

2.1.3 Mô tả bài toán của nhóm

Nhóm thực hiện giải quyết bài toán tìm đường đi ngắn nhất dựa trên các đặc điểm sau:

- **Không gian lưới 2D (2D Grid):** Bản đồ được mô hình hóa thành các ô vuông đơn vị có tọa độ (x, y) . Mỗi ô đại diện cho một trạng thái hoặc vị trí di chuyển.
- **Vật cản (Obstacles):** Các ô không thể đi qua, đại diện cho các chướng ngại vật thực tế, buộc thuật toán phải tìm các lối đi vòng.
- **Mục tiêu:** Tìm kiếm đường đi từ điểm bắt đầu (Start) đến điểm kết thúc (Goal) bằng thuật toán A*.

Việc sử dụng A* giúp tối ưu hóa thời gian tìm kiếm nhờ vào hàm Heuristic, đảm bảo tính chính xác và hiệu suất cao hơn so với các phương pháp tìm kiếm mù thông thường trong không gian lưới.

2.2 Khái quát về thuật toán A*

2.2.1 Giới thiệu thuật toán A*

Thuật toán A* (A-star) là một trong những thuật toán tìm kiếm thực thể trong đồ thị phổ biến nhất, được sử dụng rộng rãi trong các bài toán tìm đường nhờ sự kết hợp hiệu quả giữa tìm kiếm thực tế và dự báo tương lai.

Khác với các phương pháp tìm kiếm mù như Breadth-First Search (BFS) hay các thuật toán chỉ tập trung vào chi phí thấp nhất hiện tại như Dijkstra, A* thuộc nhóm thuật toán **tìm kiếm có thông tin** (Informed Search). Thuật toán này sử dụng hàm Heuristic để dự đoán hướng đi tiềm năng nhất, từ đó giúp thu hẹp đáng kể không gian tìm kiếm và tăng tốc độ xử lý mà vẫn đảm bảo tìm được đường đi tối ưu.

2.2.2 Công thức toán học

A* hoạt động bằng cách mở rộng các nút từ điểm bắt đầu dựa trên hàm đánh giá $f(n)$. Hàm này được cấu thành từ hai thành phần chính:

$$f(n) = g(n) + h(n) \quad (2.1)$$

Trong đó:

- $g(n)$: Chi phí thực tế tích lũy từ điểm bắt đầu đến nút hiện tại n .

- $h(n)$: Hàm Heuristic ước lượng chi phí từ nút hiện tại n đến điểm đích. Thành phần này giúp thuật toán "định hướng" được đích đến nằm ở đâu.
- $f(n)$: Tổng chi phí dự kiến thấp nhất của đường đi qua nút n .

2.2.3 Các hàm Heuristic

Để thuật toán hoạt động tối ưu trên lưới 2D, việc lựa chọn hàm $h(n)$ phù hợp là cực kỳ quan trọng. Dưới đây là hai hàm phổ biến nhất:

1. **Khoảng cách Manhattan:** Phù hợp khi thực thể chỉ có thể di chuyển theo 4 hướng (Lên, Xuống, Trái, Phải).

$$h(n) = |x_1 - x_2| + |y_1 - y_2| \quad (2.2)$$

2. **Khoảng cách Euclidean:** Sử dụng khi thực thể có thể di chuyển tự do theo mọi hướng (đường chim bay).

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (2.3)$$

So sánh các hàm Heuristic

Tiêu chí	Manhattan	Euclidean
Kiểu di chuyển	4 hướng (ô vuông)	Mọi hướng (tự do)
Độ phức tạp	Rất thấp (cộng/trừ)	Cao hơn (lũy thừa, căn bậc hai)
Độ tối ưu lưới	Tốt hơn cho lưới tiêu chuẩn	Tốt hơn cho môi trường liên tục
Số nút duyệt	Thường ít hơn (do $h(n)$ lớn)	Thường nhiều hơn (do $h(n)$ nhỏ)

Bảng 2.1: So sánh khoảng cách Manhattan và Euclidean

Ngoài ra, tùy vào mô hình di chuyển của robot, nhóm có thể xem xét thêm các hàm như:

- **Khoảng cách Chebyshev:** Sử dụng cho di chuyển 8 hướng với chi phí chéo bằng chi phí ngang: $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$.
- **Khoảng cách Octile:** Tối ưu nhất cho di chuyển 8 hướng trên lưới ô vuông khi chi phí đi chéo là $\sqrt{2}$.

2.2.4 Cách hoạt động của thuật toán

Các thành phần quản lý

Thuật toán A^* quản lý hai danh sách chính để điều phối việc duyệt nút:

- **Open List (Tập mở):** Lưu trữ các nút đã được phát hiện nhưng chưa được xét duyệt. Ban đầu danh sách này chỉ chứa nút xuất phát (Start).
- **Closed List (Tập đóng):** Lưu trữ các nút đã được duyệt và đánh giá xong. Thuật toán sẽ không xét lại các nút trong danh sách này để tránh vòng lặp vô tận.

Nguyên lý chọn nút và Quy trình thực hiện

Tại mỗi bước lặp, thuật toán thực hiện theo các nguyên tắc sau:

1. Chọn nút n có giá trị $f(n)$ nhỏ nhất từ Open List để mở rộng. Nếu có nhiều nút cùng giá trị f , ưu tiên nút có h nhỏ hơn để tiến gần về đích nhanh hơn.
2. Nếu nút n là đích (Goal), thuật toán kết thúc và truy hồi đường đi qua các nút cha.
3. Với mỗi nút láng giềng m của n :
 - Bỏ qua nếu m là vật cản hoặc đã nằm trong Closed List.
 - Tính toán chi phí $g(m)$ mới thông qua n . Nếu đường đi mới tốt hơn đường đi cũ (hoặc m chưa có trong Open List), cập nhật giá trị $f(m)$ và đặt n làm nút cha của m .

Chương 2 đã tập trung phân tích chi tiết nền tảng lý thuyết về thuật toán A^* và các phương pháp mô hình hóa bài toán tìm đường trên lưới không gian 2 chiều. Thông qua việc nghiên cứu cấu trúc đồ thị, hàm chi phí $f(n) = g(n) + h(n)$ và các hàm Heuristic như Manhattan hay Euclidean, chúng ta đã thấy được sự tối ưu của A^* trong việc cân bằng giữa hiệu suất tính toán và tính chính xác của kết quả.

Trong khi các hàm Heuristic cơ bản phù hợp với các môi trường di chuyển đơn giản, các biến thể như hàm Octile lại cho thấy thế mạnh vượt trội trong việc xử lý các bài toán lưới có vật cản phức tạp và cho phép di chuyển 8 hướng. Từ những kết quả nghiên cứu và phân tích lý thuyết này, nhóm đã lựa chọn thuật toán A^* kết hợp với hàm Heuristic phù hợp nhất để tiến hành xây dựng và cài đặt chương trình. Chi tiết về quá trình thiết kế hệ thống, kiến trúc mã nguồn và kết quả thực nghiệm sẽ được trình bày cụ thể trong chương tiếp theo – Chương 3.

Chương 3

THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Sau khi đã xây dựng hệ thống cơ sở lý thuyết chi tiết về thuật toán A^* và các mô hình tính toán liên quan tại Chương 2, Chương 3 đóng vai trò là giai đoạn hiện thực hóa và kiểm chứng các lý thuyết đó thông qua môi trường mô phỏng thực tế. Việc chuyển từ các công thức toán học và sơ đồ luồng sang mã nguồn cụ thể là bước đi quan trọng để đánh giá tính khả thi và hiệu quả của giải pháp mà nhóm đã lựa chọn.

Sự cần thiết của chương này nằm ở việc chứng minh khả năng giải quyết vấn đề của thuật toán A^* trong các bối cảnh thực tế khác nhau. Thông qua việc thiết lập môi trường thực nghiệm và quan sát cách thuật toán phản ứng với các loại địa hình từ đơn giản đến phức tạp, chúng ta có thể đưa ra những đánh giá khách quan về tốc độ xử lý, độ chính xác của đường đi và khả năng tối ưu hóa của cấu trúc dữ liệu đã cài đặt.

Nội dung của chương này sẽ tập trung vào các vấn đề trọng tâm sau:

- **Môi trường và thiết lập thực nghiệm:** Giới thiệu ngôn ngữ lập trình Python, thư viện đồ họa Pygame và các quy ước về không gian lưới, màu sắc để trực quan hóa dữ liệu.
- **Các kịch bản thử nghiệm:** Phân tích chi tiết hành vi của thuật toán qua ba cấp độ: không gian trống, không gian bị chia cắt và môi trường mê cung phức tạp.
- **Đánh giá kết quả chung:** Đưa ra các nhận xét định tính và định lượng về tính tối ưu của đường đi, hiệu năng của cấu trúc dữ liệu PriorityQueue kết hợp Hash Set, cũng như tính tương tác của giao diện người dùng.

Những kết quả đạt được trong chương này sẽ là minh chứng cuối cùng cho sự thành công của đề tài trong việc áp dụng thuật toán tìm kiếm có thông tin vào bài toán thực tiễn.

3.1 Môi trường và thiết lập thực nghiệm

Để tiến hành đánh giá thực tế thuật toán A* trong bài toán tìm đường, nhóm đã xây dựng một chương trình mô phỏng với các thông số cấu hình như sau:

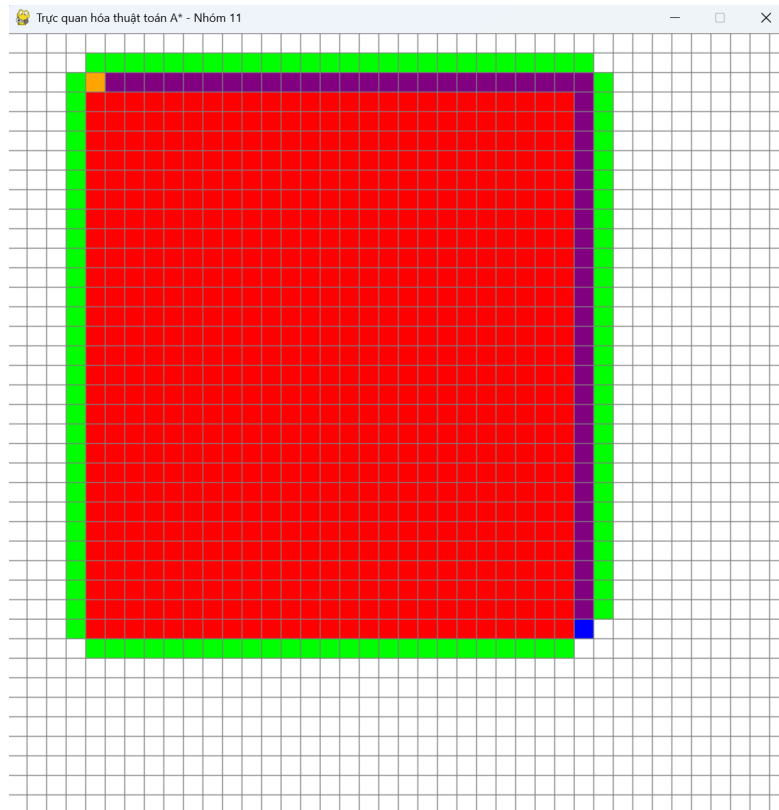
- **Ngôn ngữ lập trình:** Python.
- **Thư viện đồ họa:** Pygame (hỗ trợ vẽ không gian lưới và xử lý các sự kiện tương tác từ người dùng).
- **Kích thước không gian:** Cửa sổ hiển thị có kích thước 800x800 pixels. Không gian làm việc được chia thành một lưới 2 chiều vuông vức gồm 40x40 ô (tương đương tổng cộng 1600 Node).
- **Hàm Heuristic sử dụng:** Khoảng cách Manhattan (phù hợp với môi trường lưới di chuyển 4 hướng).
- **Quy ước hiển thị trực quan:**
 - **Màu Cam (Start):** Điểm xuất phát.
 - **Màu Xanh dương (End):** Điểm đích.
 - **Màu Đen (Barrier):** Vật cản không thể đi qua.
 - **Màu Xanh lá (Open List):** Các nút đang được đưa vào hàng đợi ưu tiên để chuẩn bị duyệt.
 - **Màu Đỏ (Closed List):** Các nút đã duyệt qua và tính toán xong chi phí.
 - **Màu Tím (Path):** Đường đi ngắn nhất cuối cùng được truy xuất.
- **Mã nguồn thực nghiệm (Source Code):** Toàn bộ mã nguồn thực nghiệm của nhóm được lưu trữ công khai và có thể truy cập tại địa chỉ: https://github.com/Vu52Hz/Thuat_Toan_A_Star.git

3.2 Các kịch bản thử nghiệm

Quá trình thực nghiệm được tiến hành qua 3 kịch bản với độ phức tạp tăng dần để đánh giá tính đúng đắn và hành vi mở rộng của thuật toán.

3.2.1 Kịch bản 1: Không gian trống (Không có vật cản)

- **Mô tả:** Đặt điểm xuất phát ở góc trên bên trái và điểm đích ở góc dưới bên phải lưới. Không vẽ thêm bất kỳ vật cản nào.
- **Thực nghiệm:** Nhấn phím SPACE để thuật toán bắt đầu chạy.

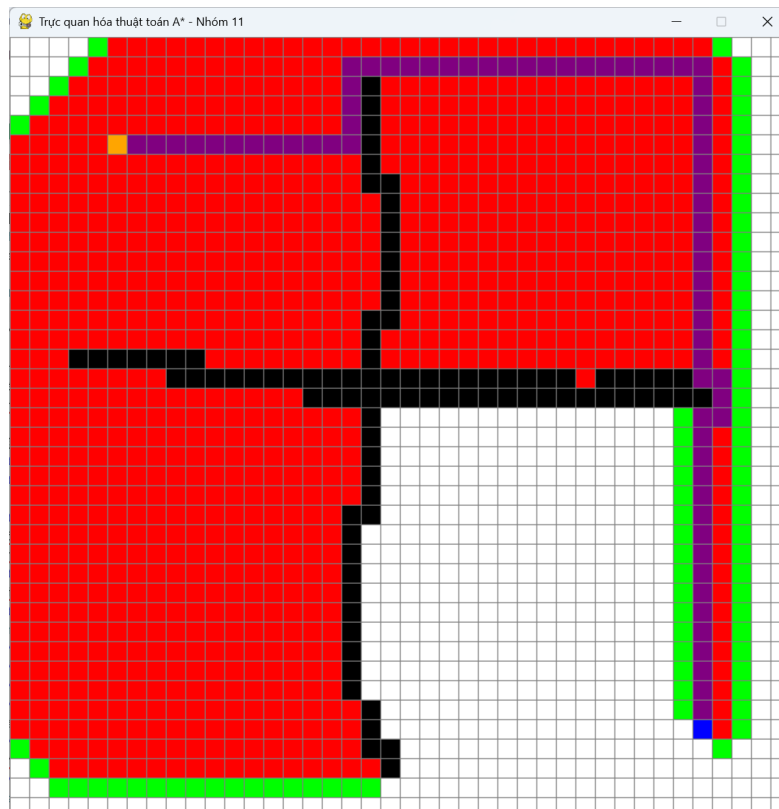


Hình 3.1: Mô phỏng thuật toán A* trong không gian trống

- **Nhận xét:** Trong không gian trống, do đặc thù của hàm Heuristic khoảng cách Manhattan, có rất nhiều đường đi khác nhau sở hữu cùng một mức chi phí tối ưu (nghĩa là giá trị $f(n)$ bằng nhau). Vì vậy, thuật toán đã tiến hành duyệt và mở rộng một vùng không gian rất lớn (phủ màu đỏ gần như toàn bộ hình chữ nhật giới hạn bởi điểm đầu và điểm cuối) để rà soát mọi khả năng trước khi chốt được đường đi. Đường đi cuối cùng được chọn (màu Tím) di chuyển men theo mép trên và mép phải của không gian.

3.2.2 Kịch bản 2: Không gian bị chia cắt bởi vật cản lớn

- **Mô tả:** Xây dựng một hệ thống tường cản dạng hình chữ thập lớn, chia cắt điểm xuất phát và điểm đích, chỉ chừa lại một lối thoát hẹp ở sát mép phải bản đồ.
- **Thực nghiệm:**

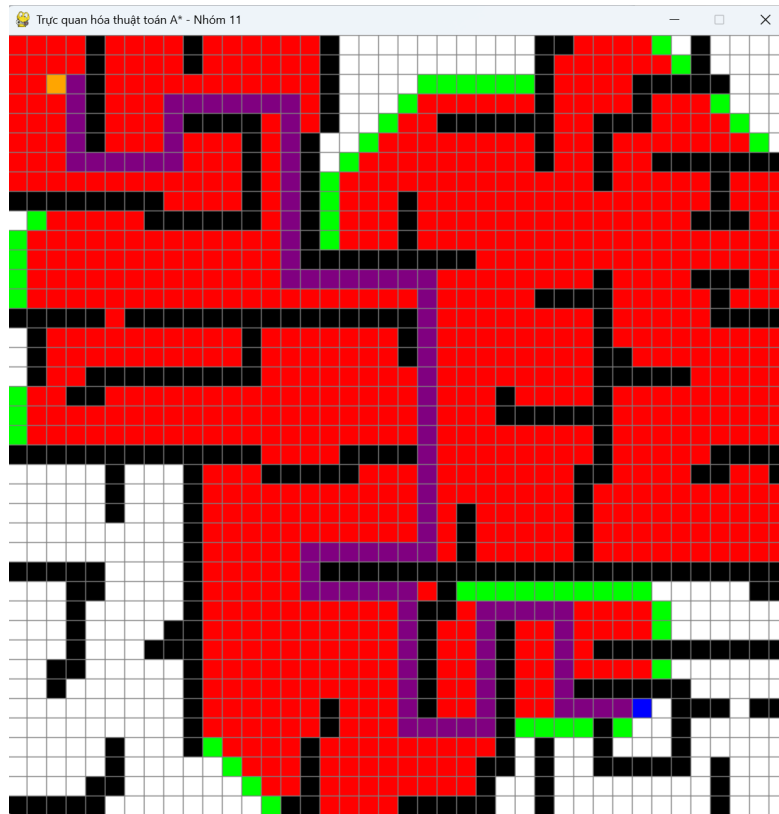


Hình 3.2: Mô phỏng thuật toán A* vượt qua vật cản lớn

- **Nhận xét:** Để vượt qua bức tường chữ thập, thuật toán buộc phải mở rộng vùng tìm kiếm (màu đỏ) lấp đầy gần như toàn bộ khu vực phía bên trái và phía trên của vật cản nhằm tìm kiếm lối rẽ. Ngay khi phát hiện ra khe hở duy nhất nằm ở hành lang sát mép phải, thuật toán lập tức hội tụ và thiết lập con đường màu Tím vòng qua rìa tường để tiến thẳng tới đích một cách chính xác.

3.2.3 Kịch bản 3: Không gian phức tạp (Dạng mê cung)

- **Mô tả:** Vẽ nhiều tường cản chằng chịt, tạo ra các đường cụt (dead-end) và các hành lang hẹp hình ziczac phức tạp giống như một mê cung.
- **Thực nghiệm:**



Hình 3.3: Mô phỏng thuật toán A* thoát khỏi mê cung

- **Nhận xét:** Dù phải đối mặt với mê cung nhiều ngõ cụt, thuật toán A* đã chứng minh được tính hiệu quả của mình. Vùng không gian đã duyệt (màu Đỏ) tuy phải lan rộng vào các nhánh phụ để rà soát, nhưng thuật toán không hề bị "lạc" hay phải duyệt toàn bộ bản đồ một cách mù quáng. Đường đi màu Tím cuối cùng được trích xuất đã len lỏi qua các khúc cua một cách hoàn hảo, hoàn toàn né tránh được các ngõ cụt để tạo thành tuyến đường ngắn nhất.

3.3 Đánh giá kết quả chung

Từ các thực nghiệm trên, nhóm rút ra các đánh giá về mặt lý thuyết cũng như cài đặt thực tế như sau:

1. Về tính chính xác và tối ưu:

Cài đặt hoàn toàn đáp ứng được tính chất cốt lõi của A*. Thuật toán luôn đảm bảo tìm ra đường đi (nếu có) và đường đi đó luôn là ngắn nhất. Việc truy vết đường đi bằng cấu trúc `came_from` hoạt động chính xác, không ghi nhận lỗi đứt gãy đường đi.

2. Về hiệu năng thuật toán:

Hiệu năng của chương trình là một điểm sáng lớn nhờ vào việc lựa chọn cấu trúc dữ liệu hợp lý trong mã nguồn:

- Việc sử dụng `PriorityQueue` giúp cho thao tác lấy ra nút có nhỏ nhất mất thời gian, cực kỳ nhanh chóng ngay cả khi số lượng nút chờ duyệt lên tới hàng ngàn.
- Đặc biệt, nhóm đã sử dụng thêm một cấu trúc Hash Set là `open_set_hash` chạy song song với Hàng đợi ưu tiên. Thao tác kiểm tra một nút đã tồn tại trong Open List hay chưa nhờ đó giảm xuống rất nhiều. Nhờ vậy, ngay cả ở "Kịch bản 3", chương trình vẫn phản hồi gần như tức thời và không có độ trễ (lag) trong khâu tính toán.

3. Về tính trực quan của Giao diện (UI):

- Chức năng chuyển đổi tọa độ chuột sang chỉ số lưới (`get_clicked_pos`) chính xác tuyệt đối. Các thao tác click trái để vẽ, click phải để xóa hoạt động trơn tru.
- Nhịp độ cập nhật hình ảnh thông qua hàm `draw()` lồng trong vòng lặp thuật toán mang lại hiệu ứng hoạt ảnh (animation) rất tốt, giúp người xem hình dung rõ ràng quá trình "suy nghĩ" và ra quyết định của thuật toán trên từng khung hình.

Chương 3 đã trình bày chi tiết quá trình thực nghiệm thuật toán A* thông qua môi trường mô phỏng Python và thư viện Pygame. Qua việc triển khai ba kịch bản thử nghiệm với độ phức tạp tăng dần, từ không gian trống đến các mô hình mê cung chằng chịt, nhóm đã chứng minh được tính đúng đắn và khả năng ứng dụng mạnh mẽ của thuật toán trong việc tìm kiếm đường đi ngắn nhất trên lưới không gian 2D.

Kết quả thực nghiệm cho thấy sự phối hợp hiệu quả giữa cấu trúc dữ liệu `PriorityQueue` và Hash Set không chỉ đảm bảo tính chính xác tuyệt

đổi về mặt toán học mà còn tối ưu hóa tối đa hiệu suất xử lý về mặt thời gian. Những quan sát trực quan về quá trình mở rộng các nút (Open List và Closed List) đã minh chứng rõ nét cho cơ chế hoạt động của hàm Heuristic Manhattan trong việc định hướng đường đi. Tổng hợp lại, toàn bộ quá trình từ nghiên cứu lý thuyết đến thực thi mã nguồn và đánh giá thực nghiệm đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu của dự án, khẳng định thuật toán A^* là giải pháp tối ưu cho bài toán tìm đường trong các môi trường có vật cản.

Chương 4

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Sau khi đã đi qua toàn bộ quá trình từ nghiên cứu lý thuyết tại Chương 2 đến triển khai thực nghiệm và đánh giá kết quả tại Chương 3, Chương 4 đóng vai trò tổng kết lại toàn bộ dự án. Đây là phần quan trọng để nhìn nhận một cách khách quan những thành tựu đã đạt được, đồng thời thẳng thắn chỉ ra những hạn chế còn tồn tại trong quá trình thực hiện đề tài.

Sự cần thiết của chương này nằm ở việc xác định giá trị thực tiễn của nghiên cứu đối với bài toán tìm đường bằng thuật toán A*. Thay vì chỉ dừng lại ở các kết quả mô phỏng, chương này sẽ giúp nhóm hệ thống hóa lại các bài học kinh nghiệm và đề xuất các giải pháp cải tiến để ứng dụng có thể vận hành tốt hơn trong các kịch bản thực tế phức tạp hơn.

4.1 Tổng kết kết quả đạt được

Sau quá trình nghiên cứu lý thuyết và tiến hành thực nghiệm, nhóm đã hoàn thành các mục tiêu đề ra ban đầu của bài tập lớn là hiện thực hóa thuật toán A* để giải quyết bài toán tìm đường trong không gian lưới hai chiều có vật cản.

Về mặt lý thuyết, đề tài đã tổng hợp và làm rõ được nguyên lý hoạt động của thuật toán tìm kiếm có thông tin, đặc biệt là vai trò của hàm chi phí $f(n)$ và hàm Heuristic (khoảng cách Manhattan) trong việc tối ưu hóa không gian duyệt để khắc phục nhược điểm duyệt rộng của các phương pháp cũ.

Về mặt ứng dụng, chương trình mô phỏng được xây dựng bằng Python và thư viện Pygame đã cung cấp một công cụ trực quan hóa sinh động, cho phép người dùng tương tác trực tiếp (như vẽ vật cản bằng chuột) và quan sát chi tiết từng bước lan tỏa từ tập mở sang tập đóng của thuật toán. Việc

kết hợp cấu trúc dữ liệu hàng đợi ưu tiên (Priority Queue) và Hash Set đã mang lại hiệu năng vượt trội, đảm bảo thuật toán phản hồi gần như tức thời và tìm ra lộ trình ngắn nhất một cách chính xác, ngay cả trong các kịch bản không gian bị chia cắt hay mê cung phức tạp. Kết quả này khẳng định tính khả thi và độ hiệu quả cao của thuật toán đã triển khai so với mục tiêu ban đầu.

4.2 Những hạn chế còn tồn tại

Mặc dù thuật toán A^* đã thể hiện hiệu suất tốt và độ ổn định cao trong các kịch bản thử nghiệm, đề tài vẫn còn tồn tại một số hạn chế nhất định cần được nhìn nhận:

- **Ràng buộc về không gian di chuyển:** Việc sử dụng hàm Heuristic Manhattan hiện tại chỉ giới hạn thực thể di chuyển theo 4 hướng cơ bản (Lên, Xuống, Trái, Phải). Điều này khiến lộ trình đôi khi mang hình dáng bậc thang cứng nhắc, chưa thực sự tối ưu và tự nhiên nếu áp dụng vào môi trường thực tế cho phép di chuyển tự do.
- **Môi trường thử nghiệm tĩnh:** Môi trường thực nghiệm hiện hành mới chỉ dừng lại ở các không gian có vật cản cố định. Chương trình chưa có cơ chế để xử lý tình huống các chướng ngại vật xuất hiện hoặc thay đổi vị trí liên tục trong quá trình thuật toán vận hành.

4.3 Hướng phát triển

Dựa trên những hạn chế đã phân tích, nhóm đề xuất một số hướng phát triển mở rộng trong tương lai nhằm nâng cấp và hoàn thiện hệ thống:

- **Tối ưu hàm Heuristic và mô hình di chuyển:** Mở rộng khả năng tìm đường của thuật toán lên 8 hướng bằng cách nghiên cứu và áp dụng các hàm khoảng cách như Chebyshev hoặc Octile. Bên cạnh đó, việc kết hợp thêm các kỹ thuật làm mịn đường đi (Path Smoothing) sẽ giúp lộ trình của thực thể trở nên linh hoạt và sát với thực tế hơn.
- **Áp dụng cho môi trường có vật cản di động:** Đây là hướng phát triển cốt lõi nhằm tiếp cận các hệ thống tự hành thực tế như robot kho bãi hay drone. Nhóm hướng tới việc tích hợp các biến thể cải tiến như D^* (Dynamic A^*) hoặc Lifelong Planning A^* (LPA*) để hệ thống có khả năng tái tính toán quỹ đạo cục bộ khi phát hiện vật cản mới mà không cần duyệt lại toàn bộ bản đồ.

- **Tùy biến môi trường đa dạng:** Cải tiến giao diện người dùng để cho phép tùy chỉnh kích thước lưới tự do, đồng thời cung cấp tính năng nhập/xuất các bản đồ phức tạp từ tệp tin bên ngoài nhằm phục vụ cho các thực nghiệm chuyên sâu và đa dạng hơn.

Chương 4 đã khép lại nội dung báo cáo bằng việc tổng kết các kết quả đạt được, đồng thời thẳng thắn nhìn nhận những hạn chế còn tồn tại trong quá trình thực hiện đề tài. Thông qua việc phân tích các rào cản về môi trường tĩnh và giới hạn trong hướng di chuyển, nhóm đã đề xuất những hướng phát triển cụ thể như ứng dụng thuật toán D^* và tối ưu hóa hàm Heuristic cho các không gian phức tạp hơn.

Nhìn chung, dự án đã hoàn thành tốt mục tiêu nghiên cứu và thực nghiệm thuật toán A^* , tạo tiền đề vững chắc cho việc phát triển các hệ thống tự hành thông minh sau này. Những kinh nghiệm rút ra từ quá trình xây dựng mã nguồn trên Python và thiết kế giao diện mô phỏng không chỉ giúp nhóm nắm vững kỹ thuật lập trình mà còn hiểu sâu sắc hơn về tư duy tối ưu hóa trong khoa học máy tính. Đây chính là hành trang quan trọng để các thành viên tiếp tục nâng cấp hệ thống và ứng dụng vào các bài toán thực tiễn đa dạng hơn trong tương lai.

Tài liệu tham khảo

- [1] Từ Minh Phương (2015). *Giáo trình Nhập môn Trí tuệ nhân tạo*. Học viện Công nghệ Bưu chính Viễn thông.
- [2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics.
- [3] Stuart Russell & Peter Norvig (2020). *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson.
- [4] Pygame Community. *Pygame Documentation*. Truy cập từ: <https://www.pygame.org/docs/>
- [5] Amit Patel. *Introduction to the A* Algorithm*. Red Blob Games. Truy cập từ: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>
- [6] Magnus Lie Hetland (2014). *Python Algorithms: Mastering Basic Algorithms in the Python Language*. Apress.
- [7] Nhóm tác giả. *Mã nguồn thực nghiệm thuật toán A-Star*. GitHub. Truy cập từ: https://github.com/Vu52Hz/Thuat_Toan_A_Star.git